Dynamic Portfolio Choice with Transaction Costs - An analysis of complexity

Victor F. Rasmussen

05/08/2025

Abstract

In the field of computational economics, it is well known that solving Dynamic Portfolio Choice models with transaction costs quickly run into the curse of dimensionality. In this paper we solve such a model with two risky stocks and a bond. We also look at ways to increase the performance of the code, by analyzing the time complexity of the solution algorithm. Inspired by recent work in the field, we build a neural network to predict the no-trade region for the portfolio weights, and argue that neural networks might play a role in future attempts at solving higher-dimensional models.

1 Introduction

In modern finance, portfolio optimization is a topic of great interest to both systematic and discretionary investors. Merton (1969 & 1971) laid the foundation for modern portfolio optimization, but similar to the Black-Scholes model for options pricing, the Merton model has several simplifying assumptions, which makes the model feasible to solve. Two critical assumptions are that asset returns are uncorrelated over time, and that there are no transaction costs. While there exists a vast literature covering portfolio optimization, solving models with both correlated returns and transaction costs, all models eventually run into the same problem: *the curse of complexity*. This "curse" happens when adding complexity to the model. In a traditional Dynamic Portfolio Choice (DPC) model, there is only one *state variable*, making the model relatively easy to solve, i.e. in a computationally feasible amount of time.
In this paper, we will not focus on the case of correlated returns, but only on the addition of (proportional) transaction costs to the model. This takes the model complexity from 1 to D state variables, D being the number of assets.
he core of the paper is the solution algorithm to the model described in the paper, using dynamic programming, and as such the main purpose of the paper is the actual solving of the model. However, when the model is solved, the time (computational) complexity of the algorithm is analyzed, and found to be sub-par, at least compared to the research frontier.
To make up for this, we use two different but similar tools to speed up the performance of code. One is using Numba to compile the Python code directly to machine code which can speed up the execution of the code, minus some overhang for compilation. The other is to use "parallelization" to allow the computer to solve multiple "states" at the same time, instead of iterating over the same for-loop, one at a time. Lastly, inspired by the work of Scheidegger et al. (2023), we lay out the theoretical framework for how a neural network trains and operates. We use this to build a simple network which is tasked with predicting the no-trade region, based on the optimal portfolio weights. This is done by converting the NTR map into a binary representation of the grid, labelling each grid point 1(0) if that point is (not) a part of the NTR. We find that the neural network is able to predict the NTR with great accuracy, with some limitations. While this approach may not be novel or useful, we were not able to find any existing literature applying neural networks to DPC models with transaction costs. Thus, using NN's could possibly serve a role in solving higher-dimensional models in the future.

2 Literature review

This paper is mainly inspired by the working paper of Scheidegger et al. "A Comprehensive Machine Learning Framework for Dynamic Portfolio Choice with Transaction Costs".
Most papers written on the subject of DPC models have used traditional dy-

namic programming methods. The problem with all of these models is that, when you want to add additional assets, they run into the curse of complexity. Thus, no matter how much one optimises the code and solution algorithm, if you want to use classical DP techniques, you will have the same problem in the end.

In Cai, Judd and Xu (2013), they apply parallelization and a specific optimization package (NPSOL) in Fortran to solve a host of different portfolio models, with various numbers of stocks, and both with and without per-period consumption. They show that it is computationally tractable to solve models up to six risky stocks plus a safe asset. It is worth noting however, that the seven asset model was solved using a supercomputer with 480 cores and still took 16 minutes to solve. Nonetheless, the paper still demonstrated that a 3 asset model could be solved in 8 minutes on a single core on a Mac laptop with a 2.5 GHz processor[1].

The results of Cai, Judd and Xu (2013) can therefore be seen as as close to the research frontier as reasonably possible, by only applying dynamic programming and software and hardware optimization.

Another approach from the research frontier comes from Schober et al. (2022). Here, instead of the usual method of constructing grids for solving the model, they utilize a technique called "sparse grids" as well as cubic B-splines. Sparse grids are used to cut down the amount of computations, as they essentially remove a lot of the grid where the density of the solutions are not required to be as fine as other places. They solve a benchmark model with five risky assets plus a bond, and use a small compute cluster to solve the model. While they do not report the time it took to solve the model, they report that when solving on the full grid, a two-asset model took nine hours to solve, and a three-asset model would take a week to solve completely.

The most recent paper on high-performance DPC model solving is by Scheidegger et al. (2023): "A comprehensive mahcine learning framework for dynamic portfolio choice with transaction costs". In this paper, the authors lay out a general framework for applying Gaussian Process Regressions (GPRs) in DP problems, in order to approximate the value and policy functions[2]. Adding to this, they innovate a technique for the approximation of the no-trade region for each iteration.

Using GPRs allows this solution method to completely sidestep any grid-based methods. This makes it possible, according to the paper, to solve models of higher dimensions than previously possible. However, they only solve a five-asset model plus a bond and per-period consumption. The authors do not mention the time spent solving the specific models, but do mention that the all experiments in the paper took under one week to solve, and the two risky asset model was "significantly faster".

[1] The work done in this paper was done on a Windows PC with 16GB RAM, 8 CPU cores, 16 threads, 3.30 GHz processor

[2] Scheidegger et al. (2023): A Compehensible Machine Learning framework for dynamic portfolio choice with transaction costs

3 Model theory

In this paper, we will be using a version of a classic benchmark model, similar to ones used in other papers, including Scheidegger et al. (2023), Schober et al. (2022), and Cai et al. (2013)

We consider an investor with a discrete and finite time-horizon, T. This is discretized into $N+1$ time periods, where $0 = t_0 < t_1 < t_2 < \cdots < t_N = T$, with equal spacing between t_i and t_{i+1} $\forall i \in N$.

Their goal is to maximize expected lifetime utility, subject to a no-borrowing and no shorting constraint. The investor has an initial endowment, which is invested into D risky assets, which may have some correlation, ρ. The investor's utility function in this model is a CRRA function:

$$u(c) = \begin{cases} \frac{c^{1-\gamma}}{1-\gamma} & \text{if } \gamma \neq 1 \\ \log(c) & \text{if } \gamma = 1 \end{cases}$$

Where γ serves as a measure of risk aversion.

In the asset market, we have D assets with stochastic returns (risky stocks), denoted, $\mathbf{R} = (R_1, \ldots, R_D)^\top$, as well as a risk-free asset (bond), R_f, which yields a fixed interest rate r per period.

The trading of the risky assets are subject to a proportional transaction costs, $\tau \in [0, 1]$, while the risk-free bond is not.

This particular model is chosen without per-period consumption. A DPC model with per-period consumption can be seen as a retirement portfolio with withdrawals, while this model is analogous to a fixed-time investment portfolio, liquidated at T. We denote the asset holdings in period t as $\mathbf{x}_t = (x_{1,t}, \ldots, x_{D,t})^\top \in [0, 1]^D$, such that each $x_{i,t}$ represents the fraction of the total portfolio allocated to asset i in period t. Each period, the investor has the opportunity to rebalance their portfolio, by buying or selling a portion of each asset. In particular, we denote the portfolio update as $\delta_t = (\delta_{1,t}, \ldots, \delta_{D,t}) \in [0, 1]^D$. If $\delta_{i,t} > 0$, it represents an increase in the portfolio weight (buying the asset), and vice versa for $\delta_{i,t} < 0$. For each transaction, the investor pays $\tau|\delta_{i,t}|$ in transaction costs, and the total transaction costs are:

$$\tau \sum_{i=1}^{D} |\delta_{i,t}|$$

What is left of the portfolio not invested in the risky assets or paid in transaction fees, are invested in the bond:

$$b_t = 1 - \mathbf{1}^\top \cdot \mathbf{x}_t - \mathbf{1}^\top \cdot \delta_t - \mathbf{1}^\top \cdot \tau|\delta_t|$$

And the state transitions are given by[3]:

$$\pi_{t+1} \equiv b_t R_f + (\mathbf{x}_t + \delta_t)^\top \cdot \mathbf{R}_t, \quad W_{t+1} = W_t \pi_{t+1}$$

[3] $\odot$ represents the Hadamard product, defined for two matrices A, B with equal dimensions $m \times n$ as the element-wise multiplication: $(\mathbf{A} \odot \mathbf{B})_{ij} = \mathbf{A}_{ij} \cdot \mathbf{B}_{ij}$, where i and j represents the row and column respectively

$$\mathbf{x}_{t+1} = \frac{(\mathbf{x}_t + \delta_t) \odot \mathbf{R}_t}{\pi_{t+1}}$$

For the investor, their goal is naturally to maximize their expected lifetime utility, and since the model is finite-horizon, we assume life ends at T, and the overall maximization problem becomes: And the Bellman equation becomes:

$$V_t(W_t, \mathbf{x}_t) = \max_{\delta_t} \mathbb{E}\{V_{t+1}(W_{t+1}, \mathbf{x}_{t+1})\} = \max_{\delta_t} \mathbb{E}\{\pi_{t+1}^{1-\gamma} V_{t+1}(\mathbf{x}_{t+1}) \mid \mathbf{x}_t\}$$

With terminal value

$$V_T(W_T, \mathbf{x}_T) = u(W_T) = u((1 - \tau \mathbf{1}^\top \cdot \mathbf{x}_T)W_T), \quad V_T(\mathbf{x}_t) = u(1 - \tau \mathbf{1}^\top \cdot \mathbf{x}_t)$$

This comes from the fact that to maximize utility, the investor needs to liquidate all of their assets, in order to consume them at the end. As mentioned above, the investor is subject to a no-borrowing and no-shorting constraint:

$$\delta_t W_t \geq -\mathbf{x}_t W_t \Leftrightarrow \delta_t \geq -\mathbf{x}_t$$

$$b_t W_t \geq 0 \Leftrightarrow b_t \geq 0$$

$$\mathbf{1}^\top \cdot \mathbf{x}_t \leq, \quad \text{for } t \in [0, T]$$

Since the utility function in question is a CRRA function, we have manipulated the terms to get a normalization and drop W as a state variable, thereby reducing the complexity of the optimization from a $D + 1$ dimensional problem to a D dimensional one. This allows us to solve for the globally optimal solution, as this solution does not depend on the level of wealth. To revert back to a specific wealth level, we would only have to multiply by W,

3.1 The no-trade region

The no-trade region is that part of the state space where the optimal portfolio allocation point lies (the Merton point), but is unable to do, because the transaction costs offset the additional benefits from doing so. Instead, the optimal will lie somewhere on the edge of the NTR, and research has shown that this region takes the shape of a square or a parallelogram[4]. In Lynch & Tan (2010), they find that investors face a 2-25% drop in utility when access to a risky asset is restricted, forcing the investors to hold the market index[5].
Mathematically, the NTR is defined as

$$\Omega_t = \{\mathbf{x}_t : \delta_{\mathbf{t}}^* = 0\}$$

where δ_t^* is the optimal policy for a given $\mathbf{x}_t$.

[4]Lynch & Tan (2010): Explaining the Magnitude of Liquidity Premia: The Roles of Return Predictability, Wealth Shocks, and State-Dependent Transaction Costs

[5]Scheidegger et al. (2023)

Figure 1: Schematic NTR

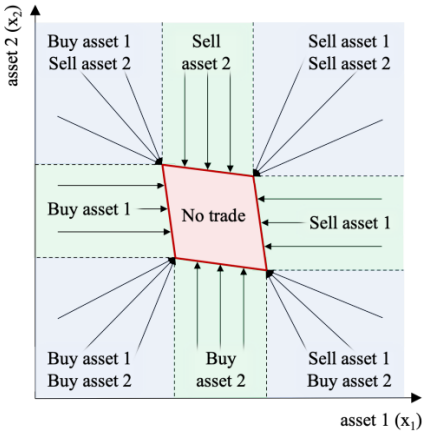


Figure 1: Schematic NTR (Scheidegger et al.)

Figure 1[6] shows the idea of what the no-trade region looks like, and what the optimal policies are, represented by arrows as proxies for δ_t^*. In the blue regions, the holdings of both assets are updates, and in the green region only one of the asset holdings are updates.

3.2 Model setup

The specific model used in this paper will be one with two risky assets, which are assumed to be identical:

$$\mu = \begin{pmatrix} 0.06 \\ 0.06 \end{pmatrix}, \quad \mathbf{\Sigma} = \begin{pmatrix} 0.2 & 0.0 \\ 0.0 & 0.2 \end{pmatrix}, \quad \rho = 0$$

We set $T = 10, \beta = 0.97, \gamma = 3.5, r = 0.03$ and $\tau = 0.01$. We fix our grids to be 100×100.

4 Dynamic Programming

We first cover the high-level theory of Bellman equations, and then move on to the specific solution algorithm employed, numerical integration and the specific coding implementation.

4.1 The Bellman operator and equation

Let $B(S)$ be the Banach space of bounded, measurable, real-valued functions on S, $f : S \to \mathbb{R}$ under the supremum norm $\|f\| = \sup_{s \in S} |f(s)|$. The Bellman

[6] Scheidegger et al. (2023

operator $\Gamma : B(S) \to B(S)$ is defined by[7]

$$\Gamma(W)(s) \equiv \max_{d \in D(s)} \left[u(s,d) + \beta \int W(s') p(ds'|s,d) \right]$$

which is the right hand side of the Bellman equation, and thus the Bellman equation $V = \Gamma(V)$ is a functional equation (mapping from value functions to value functions). V is the fixed point of Γ, i.e. the solution to $V = \Gamma(V)$.

To determine existence and uniqueness, we must impose regularity conditions: S and D are compact metric spaces, $u(s,d)$ is jointly continuous and bounded, $s \to D(s)$ is a continuous correspondence. Γ is a contraction mapping. For $\beta \in [0,1[$ and $W, V \in B(S)$ we have

$$\|\Gamma(V) - \Gamma(W)\| \leq \beta \|V - W\|$$

To show this, we can invoke the *contraction mapping theorem* and *Blackwell's theorem*[8]. The optimal Markovian decision rule δ can be approximated by the solution δ_T to a T-period problem with time-separable utility functions $U(s,d) = \sum_{t=0}^{T} \beta^t u(s_t, d_t)$.

4.2 Backwards induction

To solve the dynamic portfolio choice model, the most common solution method is to use dynamic programming (DP), and specifically *backwards induction* (a version of value function iteration). The idea in using backwards induction, is that since the terminal value function is given, we can iterate backwards in time, to get the optimal solution. In theory, the both the state and control variables in our problems are continuous. In reality, however, it is not necessarily possible to buy a fraction of a stock. Furthermore, to solve our model numerically, we will need to discretize the state space. Fortunately, our finite time horizon is already discretized as $0 = t_0 < t_1 < \cdots < t_N = T$ for $[0,T]$, but we will need to discretize the state and control space, to make the problem computationally tractable[9]. In the following, we outline the general algorithm to solve a general model with backwards induction:

[7] Dynamic Programming course at UCPH, lecture slide 1.1

[8] A detailed explanation of the contraction mapping theorem and Blackwell's theorem is deemed beyond the scope of this paper

[9] Cai, Judd & Xu (2013): Numerical Solution of Dynamic Portfolio Optimization with Transaction Costs

Algorithm 1 Finite–horizon value-function iteration on a discrete grid

Construct state grid $\mathcal{S} = \{s_1, \ldots, s_S\} \subseteq S$
for $d \leftarrow 1$ **to** D **do** ▷ D control dimensions
    Construct action grid $\mathcal{W}_d = \{w_{1,d}, \ldots, w_{N,d}\}$
Choose numerical routine Expect$(\cdot)$ for conditional expectations

for $i \leftarrow 1$ **to** S **do**
    $V_T(s_i) \leftarrow g_T(s_i)$ ▷ terminal payoff

for $t \leftarrow T-1$ **downto** 0 **do**
    for $i \leftarrow 1$ **to** S **do** ▷ loop over states
        $V_{\text{best}} \leftarrow -\infty$
        $\pi_{\text{best}} \leftarrow \varnothing$
        for all $a \in \mathcal{W}_1 \times \cdots \times \mathcal{W}_D$ **do** ▷ Cartesian product grid
            $\mathit{flow} \leftarrow u_t(s_i, a)$
            $\mathit{cont} \leftarrow \text{Expect}\big[V_{t+1}(s') \mid s_i, a\big]$
            $RHS \leftarrow \mathit{flow} + \beta \cdot \mathit{cont}$
            if $RHS > V_{\text{best}}$ **then**
                $V_{\text{best}} \leftarrow RHS$
                $\pi_{\text{best}} \leftarrow a$
        $V_t(s_i) \leftarrow V_{\text{best}}$
        $\pi_t(s_i) \leftarrow \pi_{\text{best}}$

And the more general VFI algorithm (for infinite horizon) is as follows:

$$V = \Gamma(V) \rightarrow V_{n+1} = \Gamma(V_n)$$

1. Select tolerance level, $\epsilon > 0$, and initial guess on value function V_0
2. Iterate on $V_{n+1} = \Gamma(V_n)$
3. Stop if $\|V_{n+1} - V_n\| < \epsilon$, else go to step 2

The advantages of using VFI is that we have guaranteed conversion. By using the contraction mapping property, VFI converges linearly.

4.3 Numerical integration

In the traditional Bellman equation[10], in the most general sense

$$V_t(s_t) = \max_{d_t \in D(s_t)} \Big[u_t(s_t, d_t) + \beta \int V_{t+1}(s_{t+1}) p(ds_{t+1} | s_t, d_t)\Big]$$

we have two parts of the right-hand-side of the equation, as described in the algorithm above. The $u_t(s_t, d_t)$ is obviously deterministic and poses no immediate

[10] Dynamic Programming course at UCPH, lecture slide 1.3

threat, from a computational point-of-view. However, since we cannot directly compute the integral computationally, we will again have to resort to numerical methods. There are many possible ways to do this, but we here present two(three) different numerical methods and discuss their various advantages and drawbacks associated with each method.

4.3.1 Monte Carlo simulation

The idea behind Monte Carlo methods is to draw a large number of samples from a known distribution.
The idea behind Monte Carlo methods is to draw a large number of samples from a known distribution. Let $X_1, \ldots, X_n$ be independent draws from a stochastic variable X with mean $\mu = \mathbb{E}[X]$ and variance $\sigma^2 = Var[X]$. The MC estimator of μ is our sample mean

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^{n} X_i, \quad X \sim N(\mu, \sigma^2)$$

which follows by applying the Law of Large Numbers (LLN):

$$\hat{\mu} \rightarrow^{a.s.} \mu \text{ as } n \rightarrow \infty$$

The computational complexity[11] of using Monte Carlo is $O(n)$ and the error scales by $O(n^{-1/2})$. A benefit of using MC is its simple nature and ease of understanding. Furthermore, the method is "embarrassingly parallel", meaning it is trivial to speed up the computations by running it across CPUs/GPUs, unlike most other methods, which are not necessarily parallelizable.
However, when using Monte Carlo, one drawback is the computational complexity required in order to lower the simulation error to a desired level, since a large number of draws is required, and otherwise to accept that the maximization may not be accurate.
There does however exist several techniques to lower the variance without increasing n, such as using "antithetic variates", "control variates", "importance sampling" and "common random numbers". However, discussion of these methods and their exact impact on the simulation error is considered outside the scope of this paper, and will not be discussed further.

4.3.2 Gauss-Hermite quadrature

Gauss-Hermite quadrature is a specific application of the more general Gaussian quadrature

$$\int_a^b f(x)w(x)\,dx = \sum_{i=1}^{n} \omega_i f(x_i) + \varepsilon$$

[11] Computational complexity is here described using big-O notation, a tool borrowed from computer science

where ω_i are the quadrature weights, $x_i \in [a, b]$ the quadrature nodes and $w(x)$ is the specific weighing function. For Gauss-Hermite quadrature, the weighing function is e^{-x^2} on $[-\infty, \infty]$, and the weights and nodes come from the Hermite polynomials:

$$\int_{-\infty}^{\infty} e^{-x^2} f(x)\, dx = \sum_{i=1}^{n} \omega_i f(x_i) + \varepsilon, \quad \varepsilon = \frac{n!\sqrt{\pi}}{2^n (2n)!} f^{(2n)}(\xi), \quad 0 < \xi < \infty$$

Since the weighing function matches the normal density, scaled by $\frac{1}{\sqrt{\pi}}$, Gauss-Hermite quadrature is the natural choice for the numerical evaluation of normally-distributed random variables. The nodes, $x_i,\ i = 1, \ldots, n$, are the n roots of the *physicist's* Hermite polynomial $H_n(x)$

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2}$$

and the weights are[12]

$$\omega_i = \frac{2^{n-1} n! \sqrt{\pi}}{n^2 [H_{n-1}(x_i)]^2}$$

The error scaling for Gauss-Hermite quadrature is $O(e^{-cn})$ for some $c > 0$ and the computational complexity is $O(n^d)$ where n is the number of Hermite nodes and d is the number of independent Gaussian factors. In our case, this means that $d = 2$ and $n = 5$.

In this paper and corresponding code, we use Gauss-Hermite quadrature. We do this for a number of reasons. The first is that Gauss-Hermite is the "best" choice from the family of Gaussian quadratures, since it handles normally distributed variables well, secondly due to the fact that it does not add a lot of complexity to the system, and finally because it has a good integration error.

4.3.3 Python implementation

The concrete implementation of the model in this paper is done in Python 3.12. Since Python is a high-level interpreted (rather than compiled) programming language, it does not naturally lend itself to high-level performance, compared to other languages such as C, C++ or Fortran. Python does, however, have several libraries/packages which can be used to speed up the computations.
The most popular and widely used of these is Numpy (Numerical Python), used for various vector and matrix operations.
In the code for this paper, we have also used Numpy extensively, as it significantly eases both the programming aspect, as well as the speed of the program.
Another Python library which can help us is Numba. Numba is a Just-In-Time (JIT) compiler, specifically designed for working with Numpy arrays and functions. The way we use Numba is to "decorate" a function or loop (as done in the code), and when running the code, the compiler then compiles the piece of code

[12] Abramowitz, M & Stegun I A (1972), "Handbook of Mathematical Functions"

directly to machine code "just in time", instead of ahead of time. One benefit of using Numba is that it makes it trivial to parallelize, simply by decorating the function with @nb.njit(parallel=True) and by using prange instead of range. What happens is that, instead of running the program as usual, it distributed the work load across multiple CPU cores. This works especially well for embarrassingly parallel loops, where each iteration is independent of the others, and is not reliant upon data from previous iterations.

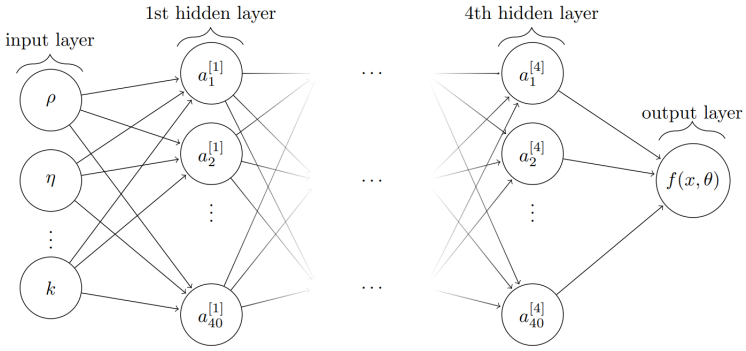
5 Neural networks

Neural[13] networks are a a type of supervised machine learning model, meaning that we have labelled training data, for what output is correct. When training the neural network, the network starts off with some initial parameter weights, which, after running the data through the model, results in some output, $\hat{y}$. This result can then be compared to the true result (labelled), and some loss/cost function can be used to correct the parameter weights to get closer to the true result. This method/training style is what is known as *backpropagation*. One of the most common *loss functions* is the Binary Cross-Entropy (BCE), defined as:

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1 - y_i)\log(p_i)]$$

where n is the number of observations, y_i is the binary label of observation i and p_i is the associated probability of the prediction. BCE loss is primarily used for binary classification tasks, such as the one described in this paper.

Figure 2: Deep neural network



[13]This part comes mainly from my previous work, writing a seminar paper in the course Advanced Finance

Pictured above is an example of a "deep" neural network[14]. What distinguishes a deep neural network from a "shallow" neural network is the number of hidden layers, i.e. the number of layers between the input and output layer. If there is more than 1 hidden layer, the neural network is considered deep.
A neural network consists of three types of layers, as illustrated above. The input layer is an N-dimensional vector, where N corresponds to the number of input variables. We then have L hidden layers, which transform the inputs from the previous layer, with a weight matrix and bias vector. Finally, the non-linear activation function is applied[15]:

$$f_l(\mathbf{a}^{[l-1]}) = g(W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{w}_0^{[l]}) = \mathbf{a}^l$$

where l is the l'th layer, and $W = (w, w_0)$ is the weights and biases.
The final layer in a neural network is the output layer. This layer is what gives the model predictions, and thus the number of output neurons corresponds to the number of possible outputs. In our model, the output is 5044-dimensional, since we want to predict an entire grid. The output layer only applies an affine transformation, and no activation function:

$$f_{L+1}(\mathbf{a}^{[L]}) = W^{[L+1]}\mathbf{a}^{[L]} + \mathbf{w}_0^{[L+1]}$$

and thus the neural network can be seen a linear regression in the last layer.
The weights and biases, W, is also the parameters the model is optimizing to minimize the loss function.
The loss function is the function that we wish to minimize. Minimizing this will give the optimal weights for the neural network. To do this, we can employ *gradient descent*. This works by computing the gradient of our loss function w.r.t. the parameters:

$$\mathbf{W}_{n+1} = \mathbf{W}_n - \gamma \nabla L(\mathbf{W}_n)$$

where $\gamma \in \mathbb{R}_+$ is the *learning rate*. This results in moving a bit closer to a local minimum.
Backpropagation is essentially applying calculus' chain rule to compute the gradient of the loss function w.r.t. each weight in the neural network. The learning rate is the hyperparameter that determines the speed of this minimization. Choosing the learning rate has large potential effects on the training, as if the learning rate is too small, the optimization can risk getting trapped in a local minimum, instead of reaching the global minimum. On the other hand, a too large learning rate can lead to the optimization never finding a minimum to descend into.
In practice, the training of the model is done in *batches* of samples from the training data, and then update the parameters from the estimate for each gradient. This type of training is what is known as *Stochastic Gradient Descent*

[14]Lind, P. P. and Gatheral, J.: "NN de-Americanization: A Fast and Efficient Calibration Method for American-Style Options", 2023
[15]Lind & Gatheral (2023): NN de-Americanization: A Fast and Efficient Calibration Method for American-Style Options

(SGD). The entire training dataset will be split up into batches of equal size, and chosen randomly. When the training process has "used up" all of the batches and updated the weights in the network, we define that as an *epoch*. There is an obvious connection between the number of epochs and accuracy of the model, but at the cost of greater compute time.

When training the neural network, we want to ensure that it is actually able to learn the "correct" function of the option price, and to do this, we mush choose an *activation function*. The activation function defines the mapping of inputs in each neuron to a certain range. We want to choose a non-linear activation function, as we can then apply the *Universal Approximation Theorem*[16], which states that if the activation function σ is continuous (and meets certain other conditions not mentioned here), then a neural network can approximate any function to an arbitrary precision, given sufficient neurons in the hidden layer.

The activation function we will be using is the *Sigmoid activation function*, defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The primary motivation for this, is the widespread use of this activation function in the literature on classification problems[17].
When the neural network has been trained, we have obtained a large set of parameters, W, that describe the model, and when predicting prices in the test set, what happens is that the model receives the input parameters, and then the neurons in the network are activated according to W and the activation function.

5.1 Hyperparameters

To choose the best configuration of our neural network, we must choose certain *hyperparameters*, which are the parameters that influences the model's training process. These are:

- Number of hidden layers, $L = 4$;
- Number of nodes in each layer, $R^{(l)} = 40$;
- Activation function $g = \sigma(x)$;
- learning rate, $\eta \in [1e-4; 1e-2]$
- batch size, $B \in \{1, 2, 4\}$
- number of epochs $E \in [100; 2000]$

[16] Hornik (1990): "*Approximation Capabilities of Multilayer Feedforward Networks*"
[17] Szandała (2020): Review and Comparison of Commonly Used Activation Functions for Deep Neural Networks

The chosen SGD optimizer in this model is ADAM, as this is the most widely used in the literature, and seems to be the one that generally performs the best. In general, choosing the correct hyperparameters is considered more of an art than a science, since there are no exact results that can tell what each hyperparameter should be. Therefore, the chosen hyperparameter space is chosen based on models applied in the existing literature. While larger neural networks generally perform better, this performance comes at a computational cost.
A possible approach for choosing hyperparameters is naive trial-and-error. We can train the network with different hyperparameters and stop the training early if we do not observe improved prediction power, measured on the test dataset. This can be seen as a grid-based approach to choosing hyperparameters.

Another possible to selecting the best network configuration is using "Bayesian Hyperparameter Optimization". Denoting the hyperparameters as $\mathcal{H}$, the Bayesian optimization assumes a prior Gaussian process for the function that maps hyperparameters, $f : \mathcal{H} \to \mathbb{R}$. The function f is evaluated on n distinct combinations of hyperparameters, which yields a Gaussian distribution over f.
We then select a new hyperparameter candidate for each attempt, by selecting the minimal value of the posterior distribution $Law(\mathcal{H})$ with upper confidence bound acquisition function[18]:

$$a_{UCB}(x;\beta) = \mu(x) - \beta\sqrt{K(x,x)}$$

where μ is the mean and K is the covariance of the posterior distribution in question. β is a parameter which balances the exploration of the search and exploitation by a tradeoff of mean and variance.

In this paper, we have chosen the easier, but sub-optimal route of trial-and-error.

6 Results

In the pictures below, we have plotted the solved model for $t = 5$. In fig. 3 we see that the NTR is approximately square, in line with theory. As a further confirmation that the program was built correctly, when comparing it to the results from Cai, Judd and Xu (2013), we find that it roughly matches their outputs, in terms of both the size, shape and location of the NTR.
In fig. 4, we have created a heatmap of the optimal policy, as a representation of the schematic NTR from earlier. We clearly see that the yellow area of the plot represents the NTR, and around it lies a simplification of the policy function.
Likewise, in fig. 5, the optimal holdings are plotted along with a proxy for the policy function for the portfolio rebalancing rules.
When analyzing more plots of the NTR (not shown here), we generally find that the NTR stays roughly in the same place, but grows steadily over time,

[18] Lind & Gatheral, 2020

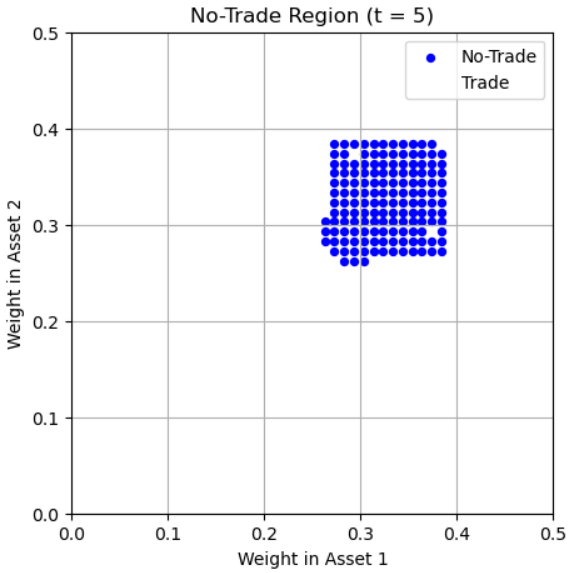


Figure 3: NTR

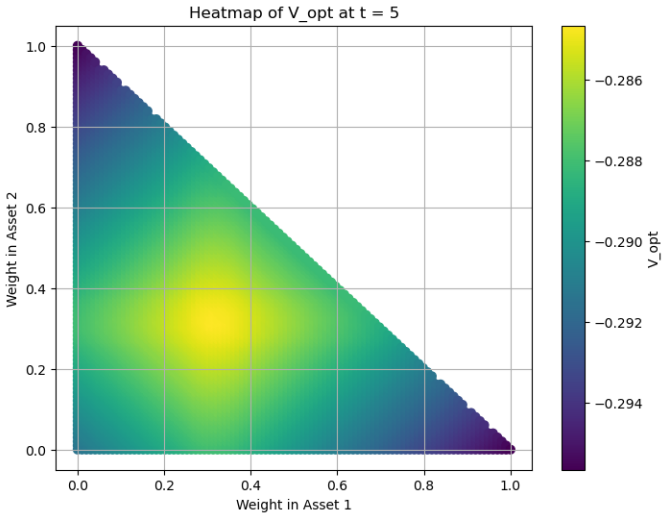


Figure 4: NTR heatmap

suggesting that as we move closer to the "end of life" (when the portfolio has to be liquidated, in order for the investor to consume), the possible portfolio rebalancing possibilities get smaller outside of the NTR. Furthermore, when trying different values of τ we also find similar NTR's, compared to the existing

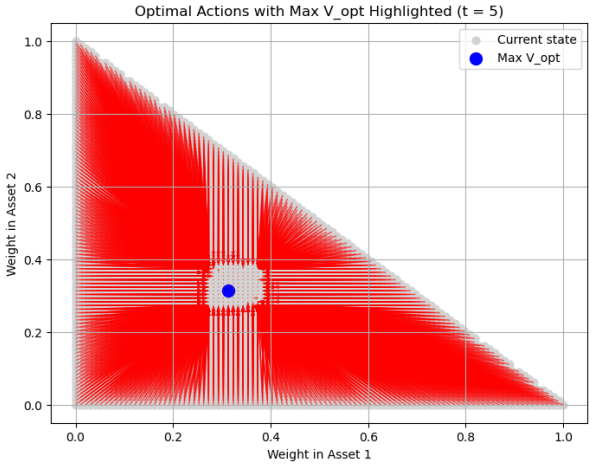


Figure 5: Proxy for NTR policy

literature.

6.1 Performance increases

When comparing the two programs (using Numba and not), we only focus on the time spent on the model solving, and not the accuracy, since both models should produce the same output for the same input.
n general, the computational complexity of the model is

$$O(T \cdot n_{grid}^2 \cdot H^d), \quad n_{grid} = \frac{(w_{grid}+1)w_{grid}}{2}$$

where w_{grid} is the number of points per row $d = 2$ is the number of risky assets and H is the number of Gauss-Hermite nodes.
The reason for this definition of w_{grid} due to the fact that when solving the model for uncorrelated, $\rho = 0$, assets with the same stochastic returns, there are no reason to ever hold more than half of one's portfolio in one asset. Thus, we only have to sample points from the $\mathbb{R}^2$-simplex (triangle), and not the entire $\mathbb{R}^2$-hypercube (unit cube). For $w_{grid} = 100$, this means that the program needs to perform $10 \cdot \left(\frac{101 \cdot 100}{2}\right)^2 \cdot 25 = 10 \cdot 5050^2 \cdot 25 = 6,375,625,000$ floating point operations. We measure the performance in flops (floating point operations per second).
Obviously, if we were to generalize the model and allow for correlation and "uneven" returns, the complexity would change such that $n_{grid} = w_{grid}^2$. If we are to generalize the complexity of this type of model where we only solve on the grid, and do not use interpolation, the complexity becomes:

$$O(T \cdot G^{2d} \cdot H^d)$$

where G is the "fineness" of the intervals. This clearly demonstrates why the approach outlined in this paper is inadequate for $d > 2$, even if using parallelization and compilation to machine code or C.

Grid size	**Time**	**Flops**
20	0.06 sec.	$1.8 \cdot 10^8$
40	0.67 sec.	$2.5 \cdot 10^8$
60	3.08 sec.	$2.7 \cdot 10^8$
80	9.71 sec.	$2.7 \cdot 10^8$
100	22.17 sec.	$2.9 \cdot 10^8$

Grid size	**Time**	**Flops**
5	0.58 sec.	$9.5 \cdot 10^4$
10	8.00 sec.	$9.4 \cdot 10^4$
20	1 min. 59.34 sec.	$9.2 \cdot 10^4$
25	4 min. 40.14 sec.	$9.4 \cdot 10^4$
30	9 min. 41.99 sec.	$9.3 \cdot 10^4$
40	30 min. 36.17 sec.	$9.2 \cdot 10^4$

From the tables we find that the program without using Numba and parallelization performs just under 100,000 flops, while the program using both performance increases does approximately 270,000,000 flops, which is a 2700X increase in performance. From this, it is quite apparent why parallelization is such an important step in modern scientific computing, when it is possible to achieve these kinds of performance enhancement.

However, this would still nok markedly impact the curse of complexity. If we assume that we still use Gauss-Hermite quadrature and a grid fineness of 100, we find that for a 3 stock model, we get an estimated required number of calculations of $1.25 \cdot 10^{15}$, and by using $3 \cdot 10^8$ flops, it would take around 48 days to solve.
What affects the model the most is obviously the grid-based solution method, hence why the research frontier is aiming to move away from grid-based solutions altogether, or at least using sparse grids to alleviate the curse of complexity. Furthermore, most modern papers as the one mentioned in this paper also use some type of interpolation. By using grid search, each linearly-spaced interval is $O(G)$, but when using interpolation, the average time complexity is $O(\log \log G)$, approximately a 65X performance increase, and using interpolation can also allow to find the exact optimum, not just the nearest grid point.

6.2 Neural network

To train the neural network, we have sampled five different $\mu's$ and five different σ's:

$$\mu = [(0.04, 0.04)^\top, (0.05, 0.05)^\top, (0.06, 0.06)^\top, (0.07, 0.07)^\top, (0.08, 0.08)^\top]$$

$$\sigma = [(0.12, 0.12)^\top, (0.15, 0.15)^\top, (0.17, 0.17)^\top, (0.20, 0.20)^\top, (0.25, 0.25)^\top]$$

This yields a training set of $5^2 \cdot 9 = 225$ training samples, as we do not care about predicting the NTR in the last period (as no rebalancing is optimal, only selling everything). We have kept T, β, γ, ρ and τ fixed, to simplify the training process of the neural network.
Thus, the inputs to the network are t, μ, σ, w_{opt} and the output is a 5044-length vector, $(x_1, \ldots, x_{5044}),\ x \in [0,1]^{5044}$. The choices for the trained network were $\eta = 1e-3, B = 4$ and $E = 1000$. Lowering the learning rate was found to simply slow down the training speed, and using a larger batch was found to be more beneficial in terms of training loss. Lastly, we initially saw a steady drop in the loss function, which slowly decreased.
After 1000 epochs, the loss was still falling, but at a significantly slower rate, and to avoid overfitting too much, we capped the training at 1000 epochs[19]. The BCE loss was 0.000701, indicative of overfitting the neural network, but this is to be expected due to the small number of training samples and the large number of epochs. From a purely output-driven point of view, this can be seen as an advantage, since our test data lies between two values of the training data, and as such, the neural network did not have to predict anything "outside" of what it had already learned.
The actual implementation of the neural network can be seen in the code, but we essentially turn the NTR plot into a binary representation for each point on the grid:

$$\begin{cases} 1 & \text{if } (w_1, w_2) \text{ is part of NTR} \\ 0 & \text{if not} \end{cases}$$

As mentioned this yields a vector with 5044 values between 0 and 1. We then use a prediction filter:

$$\begin{cases} y_i = 1 & \text{if } x_i \geq 0.5 \\ y_i = 0 & \text{if } x_i < 0.5 \end{cases}$$

and plot the results.

To find an approximation for the optimal portfolio weights (the Merton point), we found the "length" of the actual NTR and used the middle point. While this may seem like "cheating", by giving the network the solution, it still has to predict both the shape and the size of the NTR.

[19] Overfitting happens when the neural network is exposed to the same results too many times, and instead of learning the underlying pattern, it simply learns to "memorize" the result

Furthermore, it is well known that solving higher-dimensional models without transaction costs is significantly faster, and it can therefore be argued that this way of doing it is just a quick and dirty way of getting roughly the same information.

In the picture below (fig. 6), we have overlaid both the NTR that the neu-

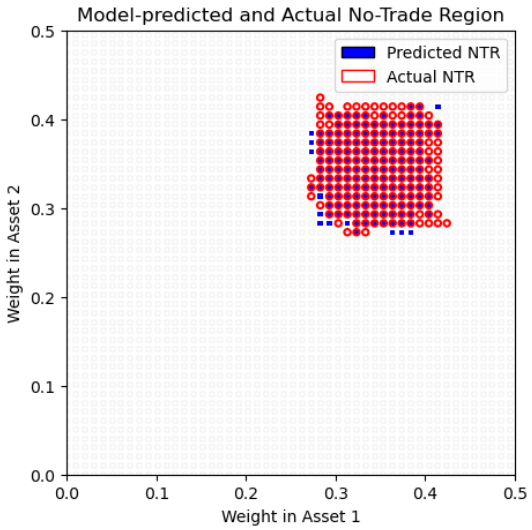


Figure 6: NN vs. DP

ral network has predicted, as well as the actual NTR from solving the model. We see an almost perfect overlap, with only slight mistakes from the neural network, which just as easily could stem from the small grid used. It is worth noting however, that we did make the job significantly easier for the NN than possible. Especially by the fixing of τ, β and γ, the network essentially did not really have to predict the shape of the NTR as much as the size. Of note also, is the fact that we actually had to solve 25 models to obtain the training data. Nonetheless, this still demonstrates that by the power of the universal approximation theorem, neural networks can be used in solving these types of models.

7 Conclusion

In this paper, we have successfully built a program to solve a dynamic portfolio choice model with transaction costs in finite horizon. We have built the model around the assumption of two risky assets plus a bond, and no per-period consumption, but consumption at the end of the life of the investor. To solve the model, we first discretized the various grids, and then used the finite-horizon

version of value function iteration, backwards induction. By knowing the optimal policy choice at T, we were able to compute policy choices and their associated portfolio weights for $T-1, T-2, \ldots, t=0$. By exhaustively searching the entire grid, we were able to completely solve the model on a relatively coarse 100×100 grid. To optimize the performance of the program and algorithm employed, we have discussed and implemented JIT compilation, from python directly to machine code, as well as parallelization, which allowed an enormous performance increase, measured in flops. We have also analyzed the time complexity of the model and looked at ways to improve it, specifically by using on-grid interpolation to get a more exact model, as well as to improve the speed of the model. We also discussed two ways of performing numerical integration, and compared Monte Carlo methods to Gauss-Hermite quadrature. We successfully implemented Gauss-Hermite quadrature based numerical integration into our model to solve the definite integrals. We chose Gauss-Hermite due to its $O(n^d)$ performance, as well as due to its theoretical ability to evaluate integrals with stochastic Gaussian variables.

To gain some perspective to the current research frontier, we became inspired by the NTR approximation algorithm from Scheidegger et al. (2023), and built a small neural network to predict the NTR based on the distribution of the stocks, as well as a proxy for the optimal portfolio weights, absent of transaction costs. We constructed a small synthetic dataset using our newly built model solver for the DPC model, and then set up and trained the neural network.
We also laid out the entire theoretical framework underpinning neural networks, and if we were to deploy a larger model, we would find more similarities with Scheidegger et al. (2023), namely that both methods involve Bayesian statistics. In their paper, they use Bayesian Active Learning (BAL), to pick evaluation spots on the grid. We would use Bayesian Hyperparameter Optimization to pick hyperparameters.
Finally, we showed that the neural network was able to accurately predict the NTR, and discussed various shortcomings of the model, but still demonstrated that NN's have potential uses in solving DPC models with transaction costs, a type of model for which we were not able to find any existing literature on, regarding the application of neural networks.

8 Discussion

While the solving of the DPC model using the python code certainly does not add any novel ideas to the research literature on the topic, our implementation of the neural network to predict the NTR, while naïve and trivial for this particular model, the approach can still serve as a starting point for solving more complex models. Even though it required solving the model 25 times to train the actual model, we still showed that just by knowing the optimal portfolio weights, the network was able to accurately predict the almost exact location of the NTR in 2d-space. When solving higher-dimensional models that cur-

rently pose a challenge, it obviously is not possible to solve it first and then use a neural network in the solution algorithm. But it might be possible to solve a smaller model, say 3-5 stocks, and then use the implied NTR shape to extrapolate the shape of a higher-dimensional NTR and thus easing the solving of yet unsolved models. It is feasible to solve high-dimensional models without transaction costs, and by knowing the optimal portfolio allocation, the neural network could be used. This question is deemed beyond the scope of this simple course paper, but is left open for future research.

Similarly, while we considered implementing the neural network into the VFI/ backwards induction solver algorithm, we deemed it too complex at this stage, but left the possibility open to try at a later date. We also considered solving the DPC model using the endogenous grid method (EGM), but since we were not sure whether it was even possible, we did not attempt it.

9 References

- Abramowitz, M & Stegun I A (1972): *Handbook of Mathematical Functions", 10th printing with corrections*, Dover
- Cai, Judd & Xu (2013): *Numerical solution of dynamic portfolio optimization with transaction costs*, NBER working paper
- Hornik, Kurt (1990): "*Approximation Capabilities of Multilayer Feedforward Networks*", Neural Networks, Volume 4, Issue 2, Pages 251-257
- Lind, Peter Pommergård & Jim Gatheral (2023): "*NN de-Americanization: A Fast and Efficient Calibration Method for American-Style Options*", SSRN
- Lynch, A. W. & Tan, S. (2010): *Multiple risky assets, transaction costs, and return predictability: Allocation rules and implications for us investors*, Journal of Financial and Quantitative Analysis, 45(4):1015–1053
- Schober, P., Valentin, J., and Pflüger, D. (2022): *Solving high-dimensional dynamic portfolio choice models with hierarchical b-splines on sparse grids*, Computational Economics, 59(1):185–224
- Scheidegger, S., Gaegauf, L. & Trojani, F. (2023): *A comprehensive machine learning framework for dynamic portfolio choice with transaction costs*, SSRN
- Szandała, T. (2021): *Review and Comparison of Commonly Used Activation Functions for Deep Neural Networks*, Studies in Computational Intelligence